

دانشگاه صنعتی شریف

دانشکده مهندسی کامپیوتر

گزارش آزمایش ۱۰

پیاده‌سازی یک پردازنده‌ی ساده

Digital Systems Design Laboratory

اعضای گروه

پوریا عرفانزاده ۴۰۱۱۱۰۹۱۸
فاطیما تیمارچی ۴۰۲۱۰۵۸۰۲
آیه صابری ۴۰۲۱۰۶۱۴۵

نام درس

آزمایشگاه طراحی سیستم‌های دیجیتال

استاد درس

دکتر علیرضا اجلالی

تابستان ۱۴۰۵

۱.۱۰ مقدمه و اهداف

هدف این آزمایش، پیاده‌سازی یک پردازنده با معماری پشته‌ای[۱] و سپس توسعهٔ آن است. مطابق دستورکار، این پردازنده پشته‌ای با هشت ثبات هشت‌بیتی دارد، مجموعه دستورش هشت دستور است، و حافظه‌ای به اندازهٔ ۲۵۶ خانهٔ هشت‌بیتی دارد که هشت خانهٔ آخر آن، یعنی آدرس‌های F8 تا FF، به‌صورت ورودی/خروجی نگاشته‌شده در حافظه[۲] به کار می‌روند.

افزون بر مجموعه دستور پایه، این آزمایش سه دستور تازه نیز می‌خواهد و برنامه‌ای که یک تابع چندبخشی[۳] را محاسبه کند. تابع خواسته‌شده چنین است:

$$Y = \begin{cases} 3X + 5, & 0 \le X < 32, \\ 2X + 20, & 32 \le X < 64, \\ X - 40, & 64 \le X \le 127. \end{cases}$$

مقدار X از ورودی برد خوانده و مقدار Y روی نمایشگرهای هفت‌قسمتی[۴] نشان داده می‌شود. اگر ورودی منفی باشد یا خروجی از محدودهٔ قابل نمایش بیرون بزند، یکی از چراغ‌های برد به نشانهٔ خطا روشن می‌گردد.

روند انجام آزمایش

ابتدا مجموعه دستور پایه و سه دستور تازه مشخّص و ریزعملیات[۵] هر یک استخراج شد. سپس پردازنده به‌صورت یک ماشین حالت متناهی[۶] نوشته شد و در کنار آن، حافظه و درگاه‌های ورودی/خروجی در یک پیمانهٔ سامانه قرار گرفتند. برنامهٔ زبان ماشین با یک همگردان[۷] کوچک به کد ماشین تبدیل شد و درستی آن پیش از هر شبیه‌سازی سخت‌افزاری، با یک شبیه‌ساز مستقل برای هر ۲۵۶ ورودی ممکن وارسی گردید. سرانجام سه میز آزمون[۸] در ModelSim اجرا و طرح در Quartus II سنتز و تحلیل زمانی شد.

[1] Stack Architecture
[2] Memory-Mapped I/O
[3] Piecewise Function
[4] Seven Segment Display
[5] Micro-operation
[6] Finite State Machine
[7] Assembler
[8] Testbench

۲.۱۰ مجموعه دستور

الف) دستورهای پایه

هشت دستور پایه همان‌هایی هستند که دستورکار تعریف کرده است. کد عملیات[۹] در چهار بیت پرارزش بایت دستور قرار می‌گیرد.

کد	دستور	طول	عملکرد
0000	PUSHC C	2	مقدار ثابت C را در پشته می‌گذارد
0001	PUSH M	2	محتوای خانه یا درگاه M را در پشته می‌گذارد
0010	POP M	2	از پشته برمی‌دارد و در خانه یا درگاه M می‌نویسد
0011	JUMP	1	از پشته برمی‌دارد و در شمارندهٔ برنامه می‌گذارد
0100	JZ	1	اگر پرچم Z یک باشد، همان کار JUMP
0101	JS	1	اگر پرچم S یک باشد، همان کار JUMP
0110	ADD	1	دو مقدار بالای پشته را جمع و نتیجه را جایگزین می‌کند
0111	SUB	1	دو مقدار بالای پشته را تفریق و نتیجه را جایگزین می‌کند

جدول ۱: هشت دستور پایه‌ی خواسته‌شده در دستورکار

دو قرارداد مهم دستورکار نیز رعایت شده‌اند: تنها دو پرچم S و Z وجود دارد و این دو تنها با اجرای ADD و SUB تغییر می‌کنند، و همهٔ محاسبات علامت‌دار و در نمایش متمّم دو[۱۰] انجام می‌شوند.

ب) سه دستور تازه

سه دستور زیر به مجموعه افزوده شدند. هر سه تک‌بایتی‌اند و عملوند جداگانه‌ای ندارند.

کد	دستور	عملکرد	اثر بر پشته و پرچم‌ها
1000	SWAP	جای دو مقدار بالای پشته را عوض می‌کند	عمق و پرچم‌ها بی‌تغییر
1001	SHL	مقدار بالای پشته را یک بیت به چپ می‌راند	عمق و پرچم‌ها بی‌تغییر
1010	CMP	$B - A$ را می‌سنجد بدون آنکه چیزی بردارد	تنها پرچم‌ها به‌روز می‌شوند

جدول ۲: سه دستور افزوده‌شده به مجموعه‌ی پایه

انگیزهٔ هر سه دستور از نیازهای همان برنامهٔ خواسته‌شده می‌آید. SHL برای ضرب در دو لازم است؛ چون مجموعهٔ پایه هیچ دستور ضربی ندارد و تابع خواسته‌شده در دو شاخهٔ خود به $2X$ و $3X$ نیاز دارد. با

[9] Opcode

[10] Two's Complement

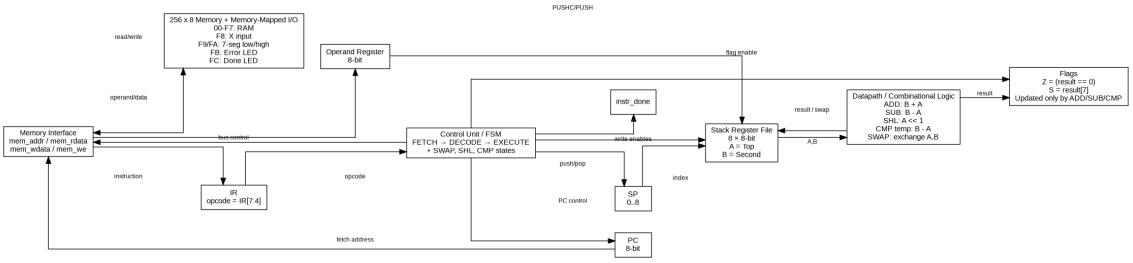
شیفت یک‌بیتی، ضرب در دو بدون هیچ بلوک ضرب‌کننده‌ای به دست می‌آید و $3X$ نیز از جمع X با $2X$ ساخته می‌شود.

CMP برای تشخیص بازه لازم است. با مجموعهٔ پایه، تنها راه مقایسه استفاده از SUB است، امّا SUB هر دو عملوند را از پشته برمی‌دارد و نتیجه را جای آن‌ها می‌گذارد؛ یعنی مقدار X که هنوز به آن نیاز داریم از میان می‌رود. CMP همان تفریق را انجام می‌دهد و تنها پرچم‌ها را به‌روز می‌کند، بی‌آنکه محتوای پشته را دست بزند.

SWAP نیز برای مرتّب کردن عملوندها در پشته پیش‌بینی شده است. در برنامهٔ نهایی به آن نیازی نیفتاد، امّا چون بخشی از توسعهٔ خواسته‌شده است، پیاده‌سازی و جداگانه آزموده شد.

۳.۱۰ نمودار بلوکی پردازنده‌ی توسعه‌یافته

پردازنده از شمارندهٔ برنامه، ثبات دستور، ثبات عملوند، پشتهٔ هشت‌تایی به‌همراه اشاره‌گر پشته، منطق محاسباتی، دو پرچم و واحد کنترل ساخته شده است. مسیر داده برای سه دستور تازه به همان پشته و همان مسیر جمع و تفریق موجود متّصل شده تا توسعه با کمترین سخت‌افزار اضافی انجام شود.



شکل ۱: نمودار بلوکی پردازنده‌ی توسعه‌یافته

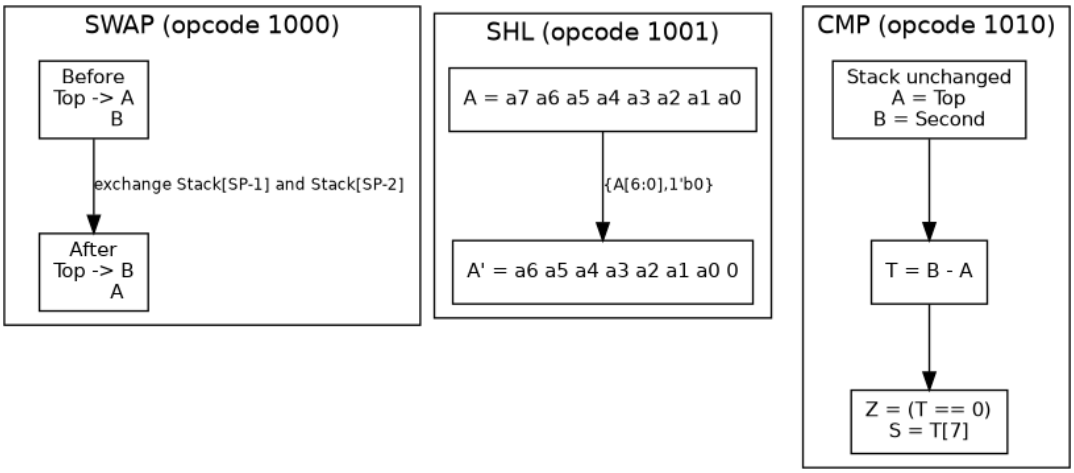
اگر A را مقدار بالای پشته و B را مقدار دوم از بالا بگیریم، روابط اصلی مسیر داده چنین‌اند:

$$A = \mathrm{Stack}[\mathrm{SP}-1], \qquad B = \mathrm{Stack}[\mathrm{SP}-2],$$

$$\text{جمع} = B + A, \qquad \text{تفریق} = B - A, \qquad \text{شیفت} = \{A[6{:}0],\, 0\}.$$

توجّه شود که هم SUB و هم CMP مقدار $B - A$ را می‌سازند، یعنی مقدار دوم منهای مقدار بالا. این ترتیب در نوشتن برنامه اهمّیّت دارد و در بخش ۶.۱۰ به آن بازمی‌گردیم.

۴.۱۰ تغییرات مسیر داده برای سه دستور تازه



شکل ۲: اثر سه دستور تازه بر پشته و پرچم‌ها

الف) SWAP

این دستور جای دو مقدار بالای پشته را عوض می‌کند. نکتهٔ پیاده‌سازی آن این است که هر دو انتساب در یک لبهٔ ساعت و به‌صورت غیرمسدودکننده[11] انجام می‌شوند؛ بنابراین سمت راست هر دو انتساب مقادیر *پیش از* لبه را می‌خواند و هیچ‌یک از دو مقدار پیش از استفاده بازنویسی نمی‌شود. اگر این دو انتساب مسدودکننده بودند، مقدار نخست پیش از خوانده شدن از بین می‌رفت و به‌جای جابه‌جایی، هر دو خانه یک مقدار می‌گرفتند. نه اشاره‌گر پشته و نه پرچم‌ها تغییر نمی‌کنند.

ب) SHL

مقدار بالای پشته یک بیت به چپ رانده می‌شود؛ بیت کم‌ارزش تازه صفر است و بیت پرارزش پیشین دور ریخته می‌شود. عمق پشته و پرچم‌ها دست‌نخورده می‌مانند. دور ریخته شدن بیت پرارزش پیامد مهمی دارد که در تحلیل برنامه به آن می‌پردازیم: اگر $2X$ از محدودهٔ هشت‌بیتی علامت‌دار بیرون بزند، بیت علامت نتیجه یک می‌شود و همین، سازوکار تشخیص خطا را فراهم می‌کند.

[11] Non-Blocking Assignment

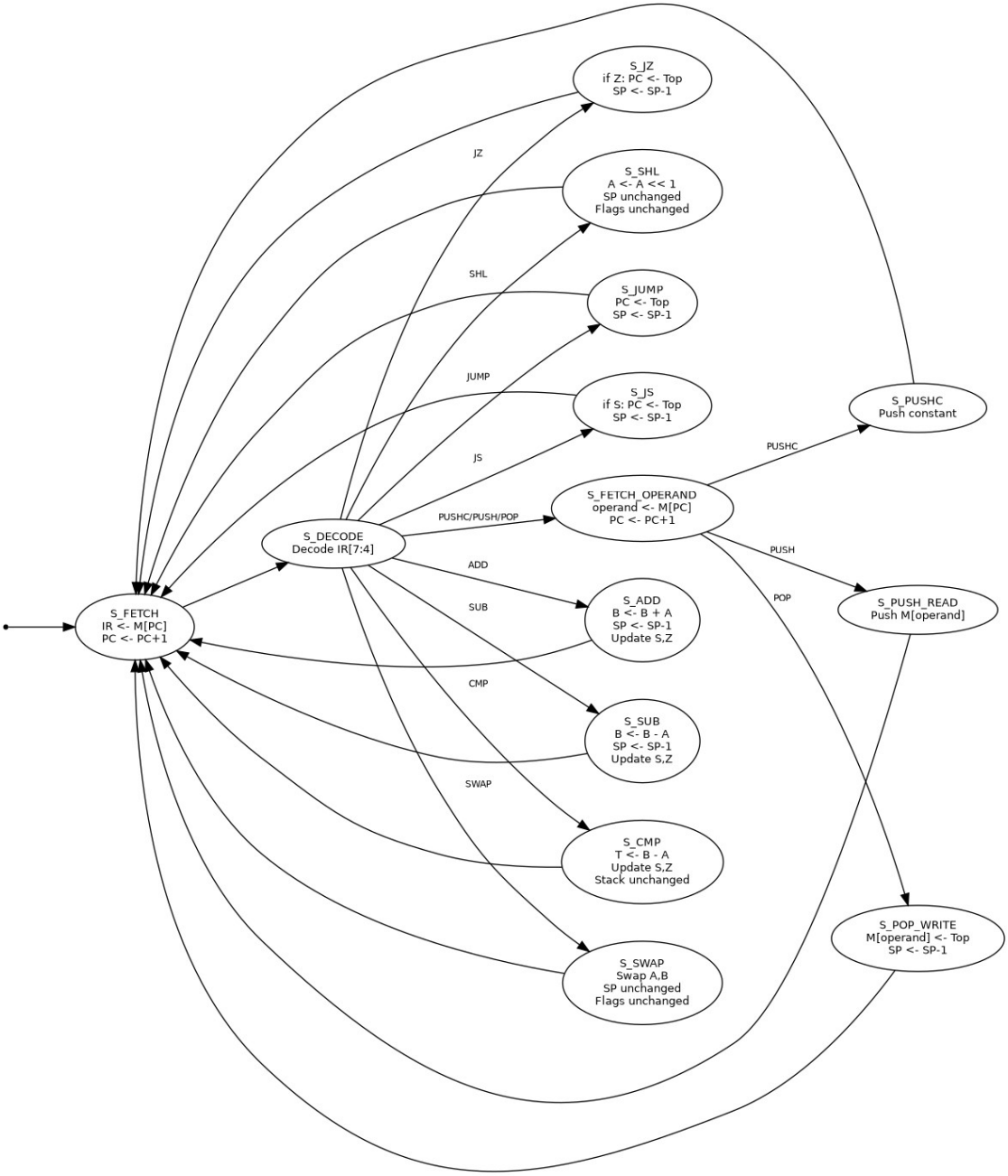
ج) CMP

مقدار موقّت $T = B - A$ روی همان مسیر تفریق موجود ساخته می‌شود، پرچم Z از صفر بودن T و پرچم S از بیت هفتم آن به دست می‌آید، و خودِ T هیچ‌جا ذخیره نمی‌شود. اشاره‌گر پشته و محتوای آن نیز تغییر نمی‌کنند. این دستور تنها دستور مجموعه است که پرچم‌ها را بدون تغییر دادن پشته به‌روز می‌کند.

۵.۱۰ واحد کنترل و ریزعملیات

واحد کنترل یک ماشین حالت متناهی با چهارده حالت است. چرخهٔ عمومی از واکشی[۱۲] به رمزگشایی[۱۳] و سپس به حالت اجرایی مناسب می‌رود و از آنجا به واکشی بازمی‌گردد. برای سه دستور تازه، سه حالت اجرایی جداگانه افزوده شده است. چون هر سه تک‌بایتی‌اند، برخلاف PUSHC، PUSH و POP وارد حالت واکشی عملوند نمی‌شوند.

[12] Fetch
[13] Decode



شکل ۳: نمودار حالت واحد کنترل به‌همراه سه حالت تازه

دستور	مرحله	ریزعملیات
هر سه	واکشی	$\mathrm{IR} \leftarrow M[\mathrm{PC}]$ ؛ $\mathrm{PC} \leftarrow \mathrm{PC} + 1$
هر سه	رمزگشایی	چهار بیت پرارزش ثبات دستور، حالت اجرایی را تعیین می‌کند
SWAP	اجرا	$\mathrm{Stack}[\mathrm{SP}-1] \leftarrow B$ ؛ $\mathrm{Stack}[\mathrm{SP}-2] \leftarrow A$
SHL	اجرا	$\mathrm{Stack}[\mathrm{SP}-1] \leftarrow \{A[6{:}0], 0\}$
CMP	اجرا	$T \leftarrow B - A$ ؛ $Z \leftarrow (T = 0)$ ؛ $S \leftarrow T[7]$
هر سه	پایان	اعلام پایان دستور و بازگشت به واکشی

جدول ۳: ریزعملیات سه دستور تازه در هر مرحله

واحد کنترل حالت‌های مرزی پشته را نیز مدیریت می‌کند. SWAP و CMP تنها هنگامی کار اصلی خود را انجام می‌دهند که دست‌کم دو مقدار در پشته باشد و SHL تنها هنگامی که دست‌کم یک مقدار باشد. اگر عمق پشته کافی نباشد، ماشین بدون هیچ دسترسی نامعتبری به واکشی بازمی‌گردد. این وارسی از خواندن خانه‌های نامعیّن پشته جلوگیری می‌کند.

```
`timescale 1ns/1ps

// 8 bit stack CPU
// opcode is stored in the high nibble in this project
module stack_cpu (
    input              clk,
    input              reset,

    output reg [7:0]    mem_addr,
    output reg [7:0]    mem_wdata,
    output reg          mem_we,
    input      [7:0]    mem_rdata,

    output reg          flag_s,
    output reg          flag_z,
    output reg          instr_done,

    output     [7:0]    debug_pc,
    output     [3:0]    debug_sp,
    output     [7:0]    debug_top,
    output     [7:0]    debug_second
);
    localparam OP_PUSHC = 4'b0000,
               OP_PUSH  = 4'b0001,
               OP_POP   = 4'b0010,
               OP_JUMP  = 4'b0011,
               OP_JZ    = 4'b0100,
               OP_JS    = 4'b0101,
               OP_ADD   = 4'b0110,
               OP_SUB   = 4'b0111,
               OP_SWAP  = 4'b1000,
               OP_SHL   = 4'b1001,
               OP_CMP   = 4'b1010;

    localparam S_FETCH         = 5'd0,
               S_DECODE        = 5'd1,
               S_FETCH_OPERAND = 5'd2,
               S_PUSHC         = 5'd3,
               S_PUSH_READ     = 5'd4,
               S_POP_WRITE     = 5'd5,
               S_JUMP          = 5'd6,
               S_JZ            = 5'd7,
               S_JS            = 5'd8,
```

```
                S_ADD           = 5'd9,
                S_SUB           = 5'd10,
                S_SWAP          = 5'd11,
                S_SHL           = 5'd12,
                S_CMP           = 5'd13;

    reg [4:0] state;
    reg [7:0] pc;
    reg [7:0] ir;
    reg [7:0] operand;
    reg [3:0] sp;
    reg [7:0] stack [0:7];
    integer i;

    wire [7:0] top_a;
    wire [7:0] second_b;
    wire [7:0] add_result;
    wire [7:0] sub_result;
    wire [7:0] shl_result;

    assign top_a       = (sp >= 1) ? stack[sp-1'b1] : 8'h00;
    assign second_b    = (sp >= 2) ? stack[sp-2'd2]  : 8'h00;
    assign add_result  = second_b + top_a;
    assign sub_result  = second_b - top_a;
    assign shl_result  = {top_a[6:0], 1'b0};

    assign debug_pc     = pc;
    assign debug_sp     = sp;
    assign debug_top    = top_a;
    assign debug_second = second_b;

    // memory bus
    always @(*) begin
        mem_addr  = 8'h00;
        mem_wdata = 8'h00;
        mem_we    = 1'b0;

        case (state)
            S_FETCH: begin
                mem_addr = pc;
            end

            S_FETCH_OPERAND: begin
                mem_addr = pc;
            end

            S_PUSH_READ: begin
                mem_addr = operand;
            end

            S_POP_WRITE: begin
                mem_addr  = operand;
                mem_wdata = top_a;
                mem_we    = (sp != 0);
            end

            default: begin
                mem_addr  = 8'h00;
                mem_wdata = 8'h00;
                mem_we    = 1'b0;
            end
        endcase
    end

    // control unit
    always @(posedge clk or posedge reset) begin
        if (reset) begin
            state      <= S_FETCH;
            pc         <= 8'h00;
            ir         <= 8'h00;
            operand    <= 8'h00;
            sp         <= 4'd0;
```

```
            flag_s     <= 1'b0;
            flag_z     <= 1'b0;
            instr_done <= 1'b0;
            for (i = 0; i < 8; i = i + 1)
                stack[i] <= 8'h00;
        end else begin
            instr_done <= 1'b0;

            case (state)
                S_FETCH: begin
                    ir    <= mem_rdata;
                    pc    <= pc + 1'b1;
                    state <= S_DECODE;
                end

                S_DECODE: begin
                    case (ir[7:4])
                        OP_PUSHC, OP_PUSH, OP_POP: state <= S_FETCH_OPERAND;
                        OP_JUMP: state <= S_JUMP;
                        OP_JZ:   state <= S_JZ;
                        OP_JS:   state <= S_JS;
                        OP_ADD:  state <= S_ADD;
                        OP_SUB:  state <= S_SUB;
                        OP_SWAP: state <= S_SWAP;
                        OP_SHL:  state <= S_SHL;
                        OP_CMP:  state <= S_CMP;
                        default: begin
                            // reserved opcode
                            state      <= S_FETCH;
                            instr_done <= 1'b1;
                        end
                    endcase
                end

                S_FETCH_OPERAND: begin
                    operand <= mem_rdata;
                    pc      <= pc + 1'b1;
                    case (ir[7:4])
                        OP_PUSHC: state <= S_PUSHC;
                        OP_PUSH:  state <= S_PUSH_READ;
                        OP_POP:   state <= S_POP_WRITE;
                        default:  state <= S_FETCH;
                    endcase
                end

                S_PUSHC: begin
                    if (sp < 8) begin
                        stack[sp] <= operand;
                        sp        <= sp + 1'b1;
                    end
                    state      <= S_FETCH;
                    instr_done <= 1'b1;
                end

                S_PUSH_READ: begin
                    if (sp < 8) begin
                        stack[sp] <= mem_rdata;
                        sp        <= sp + 1'b1;
                    end
                    state      <= S_FETCH;
                    instr_done <= 1'b1;
                end

                S_POP_WRITE: begin
                    if (sp != 0)
                        sp <= sp - 1'b1;
                    state      <= S_FETCH;
                    instr_done <= 1'b1;
                end

                S_JUMP: begin
                    if (sp != 0) begin
```

```
                pc <= top_a;
                sp <= sp - 1'b1;
            end
            state      <= S_FETCH;
            instr_done <= 1'b1;
        end

        S_JZ: begin
            if (sp != 0) begin
                if (flag_z)
                    pc <= top_a;
                sp <= sp - 1'b1;
            end
            state      <= S_FETCH;
            instr_done <= 1'b1;
        end

        S_JS: begin
            if (sp != 0) begin
                if (flag_s)
                    pc <= top_a;
                sp <= sp - 1'b1;
            end
            state      <= S_FETCH;
            instr_done <= 1'b1;
        end

        S_ADD: begin
            if (sp >= 2) begin
                stack[sp-2'd2] <= add_result;
                sp             <= sp - 1'b1;
                flag_z         <= (add_result == 8'h00);
                flag_s         <= add_result[7];
            end
            state      <= S_FETCH;
            instr_done <= 1'b1;
        end

        // B - A
        S_SUB: begin
            if (sp >= 2) begin
                stack[sp-2'd2] <= sub_result;
                sp             <= sp - 1'b1;
                flag_z         <= (sub_result == 8'h00);
                flag_s         <= sub_result[7];
            end
            state      <= S_FETCH;
            instr_done <= 1'b1;
        end

        S_SWAP: begin
            if (sp >= 2) begin
                stack[sp-1'b1] <= second_b;
                stack[sp-2'd2] <= top_a;
            end
            state      <= S_FETCH;
            instr_done <= 1'b1;
        end

        S_SHL: begin
            if (sp >= 1)
                stack[sp-1'b1] <= shl_result;
            state      <= S_FETCH;
            instr_done <= 1'b1;
        end

        // compare B-A, stack does not change
        S_CMP: begin
            if (sp >= 2) begin
                flag_z <= (sub_result == 8'h00);
                flag_s <= sub_result[7];
            end
```

```
                    state      <= S_FETCH;
                    instr_done <= 1'b1;
                end

                default: begin
                    state <= S_FETCH;
                end
            endcase
        end
    end
endmodule
```

کد ۱: پردازنده‌ی پشته‌ای با سه دستور افزوده

۶.۱۰ برنامه‌ی زبان ماشین

الف) نگاشت حافظه و درگاه‌ها

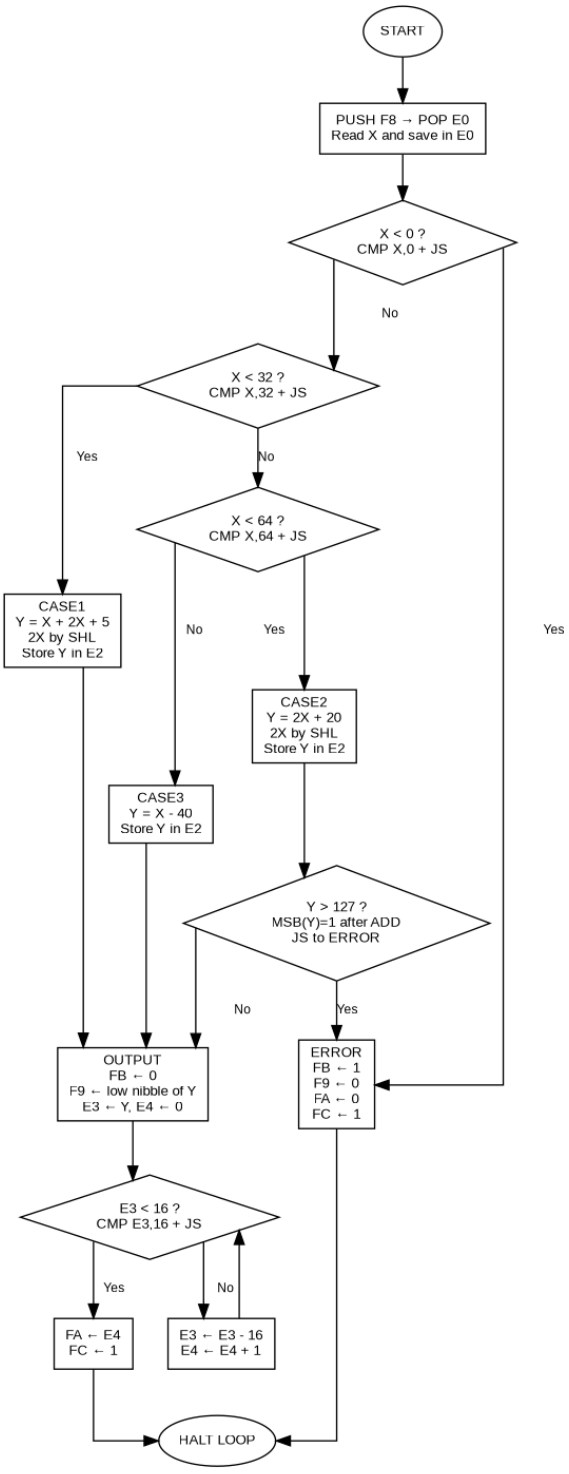
ارتباط پردازنده با ورودی و خروجی، همان‌گونه که دستورکار تصریح کرده، تنها از راه ورودی/خروجی نگاشته‌شده در حافظه انجام می‌شود. یعنی برنامه هیچ دستور ویژهٔ ورودی/خروجی ندارد و با همان PUSH و POP معمولی به درگاه‌ها دسترسی پیدا می‌کند.

آدرس	نقش	جهت
F8	مقدار ورودی X از کلیدهای برد	خواندنی
F9	رقم کم‌ارزش نمایشگر هفت‌قسمتی	نوشتنی
FA	رقم پرارزش نمایشگر هفت‌قسمتی	نوشتنی
FB	چراغ خطا	نوشتنی
FC	چراغ پایان کار	نوشتنی

جدول ۴: نگاشت درگاه‌های ورودی/خروجی در فضای حافظه

پنج خانهٔ E0 تا E4 نیز به‌عنوان متغیّرهای موقّت برنامه به کار می‌روند؛ به‌ترتیب برای نگه‌داشتن X، دور ریختن عملوندهای CMP، نگه‌داشتن Y، باقی‌مانده، و شمارندهٔ رقم پرارزش. این خانه‌ها بیرون از محدودهٔ کد برنامه‌اند و با آن تداخل ندارند.

ب) ساختار برنامه



شکل ۴: فلوچارت برنامه‌ی زبان ماشین

برنامه نخست X را از درگاه F8 می‌خواند و در E0 ذخیره می‌کند. سپس با سه مقایسهٔ پیاپی، بازهٔ ورودی تعیین می‌شود. ترتیب عملوندها در همهٔ مقایسه‌ها یکسان است: نخست X و سپس حدّ مرزی در پشته

گذاشته می‌شوند، پس $B = X$ و $A =$حد است و CMP مقدار $-X$حد را می‌سنجد. با این ترتیب، پرچم S دقیقاً هنگامی یک می‌شود که X کوچک‌تر از آن حد باشد و دستور JS می‌تواند مستقیماً برای شاخه‌بندی به کار رود.

مقایسه	شرط	مقصد
$X - 0$	$S = 1$ یعنی $X < 0$	مسیر خطا
$X - 32$	$S = 1$ یعنی $X < 32$	شاخهٔ نخست
$X - 64$	$S = 1$ یعنی $X < 64$	شاخهٔ دوم
—	در غیر این صورت	شاخهٔ سوم

جدول ۵: سه مقایسه‌ی پیاپی و شاخه‌بندی برنامه

نکتهٔ ظریف در همین جدول نهفته است. چون CMP پشته را دست نمی‌زند، پس از هر مقایسه باید دو عملوند با دو POP به خانهٔ دورریز E1 تخلیه شوند. اگر این تخلیه انجام نمی‌شد، پشتهٔ هشت‌تایی پس از چند مقایسه پر می‌شد. همین رفتار، دلیل وجود خانهٔ E1 در جدول متغیّرهاست.

ج) محاسبه‌ی سه شاخه

هر شاخه با ترکیب دستورهای موجود ساخته می‌شود:

- **شاخهٔ نخست** $3X+5$: دو نسخه از X روی پشته گذاشته می‌شود، یکی با SHL به $2X$ تبدیل می‌گردد و ADD مقدار $X + 2X = 3X$ را می‌سازد؛ سپس عدد **۵** افزوده می‌شود.
- **شاخهٔ دوم** $2X + 20$: X با SHL به $2X$ تبدیل و عدد **۲۰** به آن افزوده می‌شود.
- **شاخهٔ سوم** $X - 40$: عدد **۴۰** از X کم می‌شود.

د) تشخیص خطا

دو وضعیّت باید به روشن شدن چراغ خطا بینجامد. نخست، ورودی منفی: مقادیر 128 تا 255 در نمایش هشت‌بیتی علامت‌دار منفی‌اند و همان نخستین مقایسه با صفر آن‌ها را می‌گیرد. دوم، خروجی خارج از محدوده: این حالت تنها در شاخهٔ دوم رخ می‌دهد، چون برای X میان 54 و 63، مقدار ریاضی $2X + 20$ میان 128 و 146 می‌افتد که از 127 بزرگ‌تر است. در چنین حالتی بیت علامت نتیجهٔ هشت‌بیتی یک می‌شود و یک JS پس از محاسبه، اجرا را به مسیر خطا می‌برد.

دو شاخهٔ دیگر هرگز سرریز نمی‌کنند و این را می‌توان با محاسبهٔ ساده نشان داد: بیشینهٔ شاخهٔ نخست در $X = 31$ برابر $3(31) + 5 = 98$ و بیشینهٔ شاخهٔ سوم در $X = 127$ برابر $127 - 40 = 87$ است؛ هر دو کمتر از 127.

ه) تولید دو رقم نمایشگر

مقدار Y باید به‌صورت دو رقم شانزده‌شانزدهی روی نمایشگرها برود. رقم کم‌ارزش آسان است: سخت‌افزار درگاه تنها چهار بیت کم‌ارزش دادهٔ نوشته‌شده را به نمایشگر می‌دهد، پس نوشتن خودِ Y در F9 کافی است. رقم پرارزش دشوارتر است، چون مجموعهٔ دستور نه دستور AND دارد و نه شیفت به راست. راه‌حل به‌کاررفته، تفریق پیاپی است: در یک حلقه، مقدار ۱۶ از باقی‌مانده کم و شمارنده یکی زیاد می‌شود تا وقتی که باقی‌مانده منفی شود؛ شمارندهٔ نهایی همان $\lfloor Y/16 \rfloor$ است.

```
; main program
; E0 = X, E1 = temp, E2 = Y, E3 = remainder, E4 = high digit
; F8 input, F9 low digit, FA high digit, FB error, FC done

START:
    PUSH  F8
    POP   E0

; X < 0 ?
    PUSH  E0
    PUSHC 00
    CMP
    POP   E1
    POP   E1
    PUSHC ERROR
    JS

; X < 32 ?
    PUSH  E0
    PUSHC 20
    CMP
    POP   E1
    POP   E1
    PUSHC CASE1
    JS

; X < 64 ?
    PUSH  E0
    PUSHC 40
    CMP
    POP   E1
    POP   E1
    PUSHC CASE2
    JS

CASE3:
; Y = X - 40
    PUSH  E0
    PUSHC 28
    SUB
    POP   E2
    PUSHC OUTPUT
    JUMP

CASE1:
; Y = X + 2X + 5
    PUSH  E0
    PUSH  E0
    SHL
    ADD
    PUSHC 05
    ADD
    POP   E2
    PUSHC OUTPUT
    JUMP

```

```
CASE2:
; Y = 2X + 20
    PUSH  E0
    SHL
    PUSHC 14
    ADD
    POP   E2
    PUSHC ERROR
    JS
    PUSHC OUTPUT
    JUMP

OUTPUT:
    PUSHC 00
    POP   FB
    PUSH  E2
    POP   F9
    PUSH  E2
    POP   E3
    PUSHC 00
    POP   E4

HIGH_LOOP:
; count how many 16s are in Y
    PUSH  E3
    PUSHC 10
    CMP
    POP   E1
    POP   E1
    PUSHC HIGH_DONE
    JS

    PUSH  E3
    PUSHC 10
    SUB
    POP   E3
    PUSH  E4
    PUSHC 01
    ADD
    POP   E4
    PUSHC HIGH_LOOP
    JUMP

HIGH_DONE:
    PUSH  E4
    POP   FA
    PUSHC 01
    POP   FC
    PUSHC HALT
    JUMP

ERROR:
    PUSHC 01
    POP   FB
    PUSHC 00
    POP   F9
    PUSHC 00
    POP   FA
    PUSHC 01
    POP   FC
    PUSHC HALT
    JUMP

HALT:
; no halt instruction, stay here
    PUSHC HALT
    JUMP
```

کد ۲: برنامه‌ی زبان ماشین به زبان اسمبلی

کد ماشین این برنامه با یک همگردان کوچک تولید می‌شود که در بستهٔ تحویلی آمده است. برنامه از آدرس 00 آغاز و در 9B پایان می‌یابد. فهرست کامل کد ماشین، شامل آدرس، بایت‌ها و دستور متناظر، در همان بسته و در پروندهٔ فهرست موجود است.

۷.۱۰ حداکثر عمق پشته

یکی از خواسته‌های گزارش، تعیین بیشینهٔ عمق پشته در طول اجرای برنامه است. اهمّیّت این عدد در آن است که پشتهٔ سخت‌افزاری تنها هشت خانه دارد و سرریز آن رفتار برنامه را خراب می‌کند.
با بررسی برنامه می‌توان دید که در هیچ لحظه‌ای بیش از دو مقدار هم‌زمان روی پشته نیست: الگوی کار همواره «گذاشتن یک یا دو مقدار، انجام عملیات، برداشتن نتیجه» است و پیش از هر شاخه، پشته کاملاً تخلیه می‌شود. برای آنکه این ادّعا صرفاً استدلالی نماند، بیشینهٔ عمق در شبیه‌سازی نیز اندازه‌گیری شد و برای هر ۲۵۶ ورودی ممکن برابر **دو** به دست آمد؛ یعنی از هشت خانهٔ موجود تنها دو خانه به کار می‌رود و حاشیهٔ اطمینان چهاربرابری وجود دارد.

۸.۱۰ وارسی مستقل برنامه پیش از شبیه‌سازی سخت‌افزار

پیش از آنکه برنامه روی توصیف سخت‌افزار اجرا شود، درستی خودِ برنامه با یک شبیه‌ساز مستقل وارسی شد. این شبیه‌ساز همان مجموعه دستور را در سطح رفتاری اجرا می‌کند و برنامه را برای هر *۲۵۶ ورودی ممکن* می‌آزماید.

```
#!/usr/bin/env python3
"""small program checker"""
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
HEX = ROOT / "program" / "lab10_program.hex"


def load_memory():
    vals = [int(line.strip(), 16) for line in HEX.read_text().splitlines() if line.strip()]
    if len(vals) != 256:
        raise RuntimeError(f"wrong memory size: {len(vals)}")
    return vals


def run_case(x, max_steps=10000):
    mem = load_memory()
    ports = {0xF9: 0, 0xFA: 0, 0xFB: 0, 0xFC: 0}
    stack = []
    pc = 0
    s = 0
    z = 0
    steps = 0
    max_depth = 0
    halt_address = 0x99

    while steps < max_steps:
        if pc == halt_address and steps > 0:
```

```
            return {
                "display": ((ports[0xFA] & 0x0F) << 4) | (ports[0xF9] & 0x0F),
                "error": ports[0xFB] & 1,
                "done": ports[0xFC] & 1,
                "steps": steps,
                "max_stack_depth": max_depth,
            }

        inst = mem[pc]
        pc = (pc + 1) & 0xFF
        op = (inst >> 4) & 0x0F

        if op == 0x0:  # pushc
            stack.append(mem[pc]); pc = (pc + 1) & 0xFF
        elif op == 0x1:  # push
            addr = mem[pc]; pc = (pc + 1) & 0xFF
            if addr == 0xF8:
                stack.append(x)
            elif addr < 0xF8:
                stack.append(mem[addr])
            else:
                stack.append(0)
        elif op == 0x2:  # pop
            addr = mem[pc]; pc = (pc + 1) & 0xFF
            value = stack.pop()
            if addr in ports:
                ports[addr] = value
            elif addr < 0xF8:
                mem[addr] = value
        elif op == 0x3:  # jump
            pc = stack.pop()
        elif op == 0x4:  # jz
            target = stack.pop()
            if z:
                pc = target
        elif op == 0x5:  # js
            target = stack.pop()
            if s:
                pc = target
        elif op == 0x6:  # add
            a = stack.pop(); b = stack.pop()
            result = (b + a) & 0xFF
            stack.append(result)
            z = int(result == 0)
            s = (result >> 7) & 1
        elif op == 0x7:  # sub = B-A
            a = stack.pop(); b = stack.pop()
            result = (b - a) & 0xFF
            stack.append(result)
            z = int(result == 0)
            s = (result >> 7) & 1
        elif op == 0x8:  # swap
            stack[-1], stack[-2] = stack[-2], stack[-1]
        elif op == 0x9:  # shl
            stack[-1] = (stack[-1] << 1) & 0xFF
        elif op == 0xA:  # cmp
            a = stack[-1]; b = stack[-2]
            result = (b - a) & 0xFF
            z = int(result == 0)
            s = (result >> 7) & 1
        else:
            pass  # reserved

        max_depth = max(max_depth, len(stack))
        if len(stack) > 8:
            raise RuntimeError(f"stack overflow: x={x} pc={pc:02X}")
        steps += 1

    raise RuntimeError(f"timeout: x={x} pc={pc:02X}")


def expected(x):
```

```
    if x >= 128:
        return 0, 1
    if x < 32:
        y = 3*x + 5
    elif x < 64:
        y = 2*x + 20
    else:
        y = x - 40
    if y > 127:
        return 0, 1
    return y, 0


def main():
    mandatory = [0, 31, 32, 53, 54, 63, 64, 127, 128, 255]
    print("program check")
    for x in mandatory:
        result = run_case(x)
        exp_display, exp_error = expected(x)
        ok = (result["display"] == exp_display and result["error"] == exp_error and result["done"]
     == 1)
        print(f"X={x:3d} display={result['display']:02X} error={result['error']}
     done={result['done']} "
              f"depth={result['max_stack_depth']}  {'ok' if ok else 'error'}")
        if not ok:
            raise SystemExit(1)

    global_depth = 0
    for x in range(256):
        result = run_case(x)
        exp_display, exp_error = expected(x)
        if (result["display"], result["error"], result["done"]) != (exp_display, exp_error, 1):
            raise SystemExit(f"wrong result at x={x}")
        global_depth = max(global_depth, result["max_stack_depth"])

    print("\nall 256 inputs checked")
    print(f"max stack depth = {global_depth}")


if __name__ == "__main__":
    main()
```

کد ۳: شبیه‌ساز مستقل وارسی برنامه

ارزش این گام در جدا کردن دو گونه خطاست. اگر نتیجه‌ای نادرست باشد، این وارسی می‌گوید که مشکل از خودِ برنامه است یا از توصیف سخت‌افزار؛ چون اگر شبیه‌ساز مستقل درست جواب بدهد و سخت‌افزار ندهد، ایراد در توصیف سخت‌افزار است و برعکس. خروجی این وارسی، درستی هر ۲۵۶ ورودی و بیشینهٔ عمق پشتهٔ برابر دو را تأیید می‌کند.

۹.۱۰ میزهای آزمون و اجرا در ModelSim

سه میز آزمون خودآزما نوشته شد که هر یک لایهٔ متفاوتی از طرح را می‌سنجد.

میز آزمون	آنچه سنجیده می‌شود
دستورهای تازه	رفتار SWAP، SHL و CMP به‌صورت جداگانه
کنترل JZ	پرش شرطی بر پایه‌ی پرچم Z
برنامه‌ی نهایی	اجرای کامل برنامه برای ده ورودی نماینده

جدول ۶: سه میز آزمون آزمایش

```
# run the three tests
do run_new_instructions.do
quit -sim
do run_jz.do
quit -sim
do run_program.do
```

کد ۴: پرونده‌ی فرمان اجرای همه‌ی آزمون‌ها

الف) آزمون سه دستور تازه

```
ModelSim> do run_new_instructions.do
# if {[file exists work]} { vdel -lib work -all }
# vlib work
# vmap work work
# Copying C:\altera\13.0sp1\modelsim_ase\win32aloem/../modelsim.ini to modelsim.ini
# Modifying modelsim.ini
# ** Warning: Copied C:\altera\13.0sp1\modelsim_ase\win32aloem/../modelsim.ini to modelsim.ini.
#          Updated modelsim.ini.
#
# vlog ../rtl/stack_cpu.v
# Model Technology ModelSim ALTERA vlog 10.1d Compiler 2012.11 Nov  2 2012
# -- Compiling module stack_cpu
#
# Top level modules:
#       stack_cpu
# vlog ../tb/tb_new_instructions.v
# Model Technology ModelSim ALTERA vlog 10.1d Compiler 2012.11 Nov  2 2012
# -- Compiling module tb_new_instructions
#
# Top level modules:
#       tb_new_instructions
#
# vsim -voptargs=+acc work.tb_new_instructions
# vsim -voptargs=+acc work.tb_new_instructions
# Loading work.tb_new_instructions
# Loading work.stack_cpu
# view wave
# .main_pane.wave.interior.cs.body.pw.wf
```

شکل ۵: اجرای آزمون سه دستور تازه

```
# add wave -radix hex sim:/tb_new_instructions/mem_rdata
# add wave -radix hex sim:/tb_new_instructions/mem_wdata
# add wave sim:/tb_new_instructions/mem_we
# run -all
# new instructions test
# swap: ok
# shl 0 -> 0  ok
# shl 1 -> 2  ok
# shl 63 -> 126  ok
# shl 127 -> 254  ok
# cmp b=5 a=7: s=1 z=0  ok
# cmp b=5 a=5: s=0 z=1  ok
# cmp b=7 a=5: s=0 z=0  ok
# new instructions test: ok
# ** Note: $finish    : ../tb/tb_new_instructions.v(183)
#    Time: 3260 ns  Iteration: 1  Instance: /tb_new_instructions
# 1
# Break in Module tb_new_instructions at ../tb/tb_new_instructions.v line 183
# Simulation Breakpoint: 1
# Break in Module tb_new_instructions at ../tb/tb_new_instructions.v line 183
# MACRO ./run_new_instructions.do PAUSED at line 29
```

شکل ۶: خروجی متنی آزمون سه دستور تازه

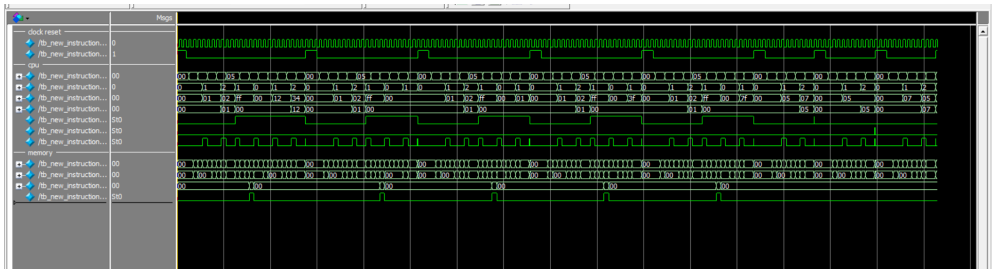
این آزمون هشت وارسی دارد. یک وارسی برای SWAP، چهار وارسی برای SHL با مقادیر مرزی، و سه وارسی برای CMP در هر سه حالت ممکن مقایسه. جدول زیر نتیجهٔ هر یک را با دلیل آن می‌آورد.

دستور	ورودی	نتیجه	چرا
SHL	0	0	شیفت صفر همچنان صفر است
SHL	1	2	ضرب در دو
SHL	63	126	بزرگ‌ترین ورودی بدون سرریز
SHL	127	254	بیت پرارزش دور ریخته می‌شود؛ نتیجه در متمّم دو منفی است
CMP	$B = 5, A = 7$	$S = 1, Z = 0$	$5 - 7 < 0$
CMP	$B = 5, A = 5$	$S = 0, Z = 1$	$5 - 5 = 0$
CMP	$B = 7, A = 5$	$S = 0, Z = 0$	$7 - 5 > 0$

جدول ۷: وارسی‌های آزمون سه دستور تازه و دلیل هر نتیجه

وارسی چهارم SHL از همه گویاتر است. ورودی 127 بزرگ‌ترین عدد مثبت هشت‌بیتی است و شیفت آن مقدار 254 می‌دهد که در متمّم دو برابر -2 است. یعنی همان سازوکاری که در بخش ۶.۱۰ برای تشخیص سرریز شاخهٔ دوم به کار رفت، اینجا به‌صورت جداگانه آزموده شده است.

ب) آزمون پرش شرطی



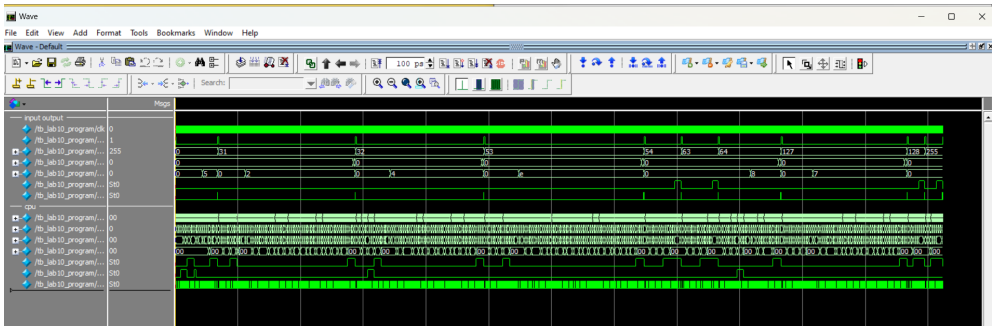
شکل ۷: شکل موج آزمون پرش شرطی JZ

```
VSIM(paused)> do run_jz.do
# JZ test
# transcript on
# if {[file exists work]} { vdel -lib work -all }
# vlib work
# vmap work work
# Modifying modelsim.ini
#
# vlog ../rtl/stack_cpu.v
# Model Technology ModelSim ALTERA vlog 10.1d Compiler 2012.11 Nov  2 2012
# -- Compiling module stack_cpu
#
# Top level modules:
#       stack_cpu
# vlog ../tb/tb_jz_control.v
# Model Technology ModelSim ALTERA vlog 10.1d Compiler 2012.11 Nov  2 2012
# -- Compiling module tb_jz_control
#
# Top level modules:
#       tb_jz_control
# vsim -voptargs=+acc work.tb_jz_control
# vsim -voptargs=+acc work.tb_jz_control
# Loading work.tb_jz_control
# Loading work.stack_cpu
# run -all
# jz: ok
# ** Note: $finish    : ../tb/tb_jz_control.v(75)
#    Time: 720 ns  Iteration: 1  Instance: /tb_jz_control
# 1
# Break in Module tb_jz_control at ../tb/tb_jz_control.v line 75
# Simulation Breakpoint: 1
# Break in Module tb_jz_control at ../tb/tb_jz_control.v line 75
# MACRO ./run_jz.do PAUSED at line 10
```

شکل ۸: خروجی متنی آزمون JZ

این آزمون کوچک، درستی مسیر پرش شرطی را جدا از بقیهٔ طرح می‌سنجد؛ یعنی بررسی می‌کند که با یک بودن پرچم Z، مقدار بالای پشته واقعاً در شمارندهٔ برنامه بنشیند و عمق پشته یک واحد کم شود.

ج) آزمون برنامه‌ی نهایی



شکل ۹: شکل موج اجرای کامل برنامه

```
VSIM(paused)> do run_program.do
# final program test
# transcript on
# if {[file exists work]} { vdel -lib work -all }
# vlib work
# vmap work work
# Modifying modelsim.ini
#
# vlog ../rtl/stack_cpu.v
# Model Technology ModelSim ALTERA vlog 10.1d Compiler 2012.11 Nov  2 2012
# -- Compiling module stack_cpu
#
# Top level modules:
#       stack_cpu
# vlog +incdir+../rtl ../rtl/lab10_system.v
# Model Technology ModelSim ALTERA vlog 10.1d Compiler 2012.11 Nov  2 2012
# -- Compiling module lab10_system
#
# Top level modules:
#       lab10_system
# vlog ../tb/tb_lab10_program.v
# Model Technology ModelSim ALTERA vlog 10.1d Compiler 2012.11 Nov  2 2012
# -- Compiling module tb_lab10_program
#
# Top level modules:
#       tb_lab10_program
#
```

شکل ۱۰: خروجی متنی اجرای برنامه برای ده ورودی

ده ورودی این آزمون تصادفی نیستند و هر یک برای پوشاندن یک وضعیّت مشخّص انتخاب شده‌اند. جدول زیر همهٔ آن‌ها را با محاسبهٔ دستی و نتیجهٔ مشاهده‌شده می‌آورد.

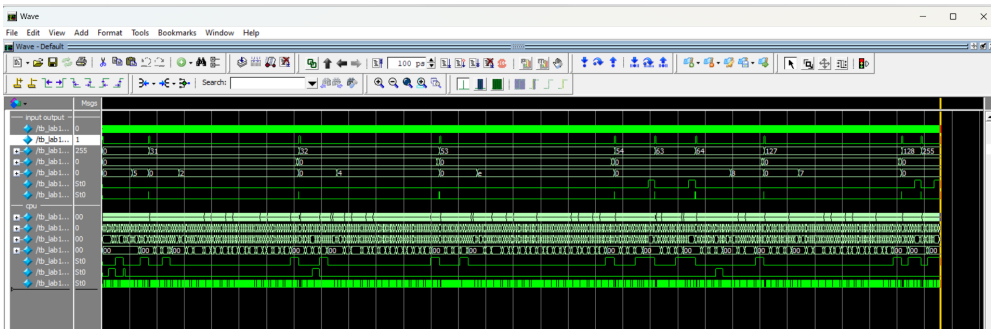
X	شاخه	محاسبه	Y	نمایش	هدف وارسی
0	نخست	$3(0)+5$	5	05	کمترین ورودی معتبر
31	نخست	$3(31)+5$	98	62	آخرین مقدار شاخهٔ نخست
32	دوم	$2(32)+20$	84	54	نخستین مقدار شاخهٔ دوم
53	دوم	$2(53)+20$	126	$7E$	آخرین مقدار بدون سرریز
54	دوم	$2(54)+20$	128	خطا	نخستین سرریز
63	دوم	$2(63)+20$	146	خطا	آخرین مقدار شاخهٔ دوم
64	سوم	$64-40$	24	18	نخستین مقدار شاخهٔ سوم
127	سوم	$127-40$	87	57	بیشترین ورودی معتبر
128	—	ورودی منفی	—	خطا	نخستین مقدار منفی
255	—	ورودی منفی	—	خطا	-1 در متمّم دو

جدول ۸: ده ورودی آزمون برنامه، محاسبه‌ی دستی و هدف هر وارسی

انتخاب این ده مقدار پوشش کاملی می‌سازد. سه جفت $(31,32)$، $(53,54)$ و $(63,64)$ دقیقاً روی مرزهای تصمیم قرار دارند و هر جفت، دو سوی یک مرز را می‌آزماید؛ اگر شرط مقایسه‌ای یک واحد جابه‌جا نوشته شده بود، دست‌کم یکی از این جفت‌ها آن را آشکار می‌کرد. دو مقدار 128 و 255 نیز مسیر ورودی منفی را می‌پوشانند.

```
# add wave sim:/tb_lab10_program/led_done
# add wave -divider {cpu}
# add wave -radix hex sim:/tb_lab10_program/debug_pc
# add wave -radix unsigned sim:/tb_lab10_program/debug_sp
# add wave -radix hex sim:/tb_lab10_program/debug_top
# add wave -radix hex sim:/tb_lab10_program/debug_second
# add wave sim:/tb_lab10_program/debug_s
# add wave sim:/tb_lab10_program/debug_z
# add wave sim:/tb_lab10_program/debug_instr_done
# run -all
# program test
# x=0   out=05  err=0  ok
# x=31  out=62  err=0  ok
# x=32  out=54  err=0  ok
# x=53  out=7e  err=0  ok
# x=54  out=00  err=1  ok
# x=63  out=00  err=1  ok
# x=64  out=18  err=0  ok
# x=127  out=57  err=0  ok
# x=128  out=00  err=1  ok
# x=255  out=00  err=1  ok
# program test: ok
# ** Note: $finish    : ../tb/tb_lab10_program.v(114)
#    Time: 61200 ns  Iteration: 1  Instance: /tb_lab10_program
# 1
# Break in Module tb_lab10_program at ../tb/tb_lab10_program.v line 114
# Simulation Breakpoint: 1
# Break in Module tb_lab10_program at ../tb/tb_lab10_program.v line 114
# MACRO ./run_program.do PAUSED at line 30
```

شکل ۱۱: ادامه‌ی خروجی اجرای برنامه



شکل ۱۲: شکل موج اجرای همه‌ی آزمون‌ها پشت سر هم

```
cd {D:/Summer/DSD/AZ 10/modelsim}
ModelSim> do run_all.do
# if {[file exists work]} { vdel -lib work -all }
# vlib work
# vmap work work
# Copying C:\altera\13.0sp1\modelsim_ase\win32aloem/../modelsim.ini to modelsim.ini
# Modifying modelsim.ini
# ** Warning: Copied C:\altera\13.0sp1\modelsim_ase\win32aloem/../modelsim.ini to modelsim.ini.
#          Updated modelsim.ini.
#
# vlog ../rtl/stack_cpu.v
# Model Technology ModelSim ALTERA vlog 10.1d Compiler 2012.11 Nov  2 2012
# -- Compiling module stack_cpu
#
# Top level modules:
#       stack_cpu
# vlog ../tb/tb_new_instructions.v
# Model Technology ModelSim ALTERA vlog 10.1d Compiler 2012.11 Nov  2 2012
# -- Compiling module tb_new_instructions
#
# Top level modules:
#       tb_new_instructions
#
# vsim -voptargs=+acc work.tb_new_instructions
# vsim -voptargs=+acc work.tb_new_instructions
# Loading work.tb_new_instructions
```

شکل ۱۳: گزارش پایانی اجرای همه‌ی آزمون‌ها

د) تحلیل شکل موج اجرای برنامه

در شکل موج اجرای کامل، چند نشانهٔ کلیدی قابل ردیابی است. شمارندهٔ برنامه از صفر آغاز می‌شود و در بازه‌های مربوط به مقایسه‌ها به‌صورت پلّکانی بالا می‌رود؛ سپس بسته به بازهٔ ورودی، به یکی از سه آدرس شاخه می‌پرد. این پرش در نما به‌صورت یک جهش ناگهانی در مقدار شمارندهٔ برنامه دیده می‌شود و نه یک پلّهٔ یک‌واحدی.

اشاره‌گر پشته در سراسر اجرا میان صفر و دو نوسان می‌کند و هرگز از دو بالاتر نمی‌رود؛ همان نتیجه‌ای که در بخش ۷.۱۰ به‌صورت عددی گزارش شد. دو پرچم S و Z تنها در لحظه‌هایی تغییر می‌کنند که یک CMP، ADD یا SUB اجرا شده باشد و در طول دستورهای PUSH و POP کاملاً ثابت می‌مانند؛ این همان قید دستورکار است که پرچم‌ها تنها با دستورهای حسابی تغییر کنند.

سرانجام، خروجی‌های درگاه در انتهای اجرا مقدار می‌گیرند: دو رقم نمایشگر هم‌زمان با نوشتن در F9 و FA به‌روز می‌شوند و چراغ پایان کار در آخرین دستور برنامه روشن می‌گردد. در ورودی‌های خطا، به‌جای آن، چراغ خطا روشن و هر دو رقم نمایشگر صفر می‌مانند.

```
# add wave -radix hex sim:/tb_new_instructions/mem_wdata
# add wave sim:/tb_new_instructions/mem_we
# run -all
# new instructions test
# swap: ok
# shl 0 -> 0  ok
# shl 1 -> 2  ok
# shl 63 -> 126  ok
# shl 127 -> 254  ok
# cmp b=5 a=7: s=1 z=0  ok
# cmp b=5 a=5: s=0 z=1  ok
# cmp b=7 a=5: s=0 z=0  ok
# new instructions test: ok
# if {[file exists work]} { vdel -lib work -all }
# vlib work
# vmap work work
# Modifying modelsim.ini
#
# vlog ../rtl/stack_cpu.v
# Model Technology ModelSim ALTERA vlog 10.1d Compiler 2012.11 Nov  2 2012
# -- Compiling module stack_cpu
```

شکل ۱۴: نمای نزدیک خروجی درگاه‌ها در پایان اجرا

```
# add wave -radix hex sim:/tb_lab10_program/debug_second
# add wave sim:/tb_lab10_program/debug_s
# add wave sim:/tb_lab10_program/debug_z
# add wave sim:/tb_lab10_program/debug_instr_done
# run -all
# program test
# x=0   out=05  err=0  ok
# x=31  out=62  err=0  ok
# x=32  out=54  err=0  ok
# x=53  out=7e  err=0  ok
# x=54  out=00  err=1  ok
# x=63  out=00  err=1  ok
# x=64  out=18  err=0  ok
# x=127  out=57  err=0  ok
# x=128  out=00  err=1  ok
# x=255  out=00  err=1  ok
# program test: ok

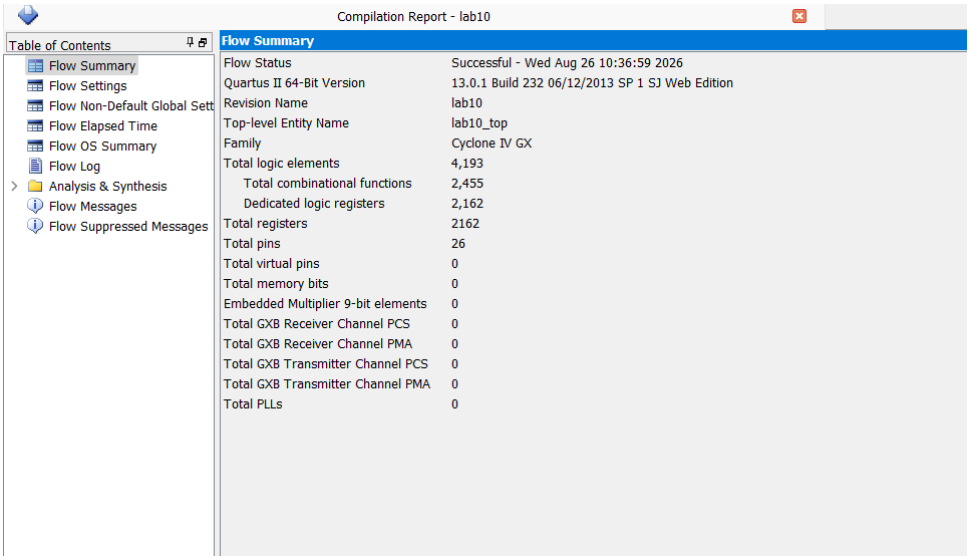
VSIM 3> do run_all.do
```

شکل ۱۵: نمای نزدیک مسیر خطا

۱۰.۱۰ سنتز و تحلیل زمانی

الف) مصرف منابع

طرح در Quartus II 13.0.1 و برای خانوادهٔ Cyclone IV GX همگردانی شد. انتخاب قطعه روی حالت خودکار بود و ابزار قطعهٔ EP4CGX15BF14C6 را برگزید. نتیجه 3,208 عنصر منطقی شامل 2,455 تابع ترکیبی و 2,162 ثبات اختصاصی، و ۲۶ پایه است.

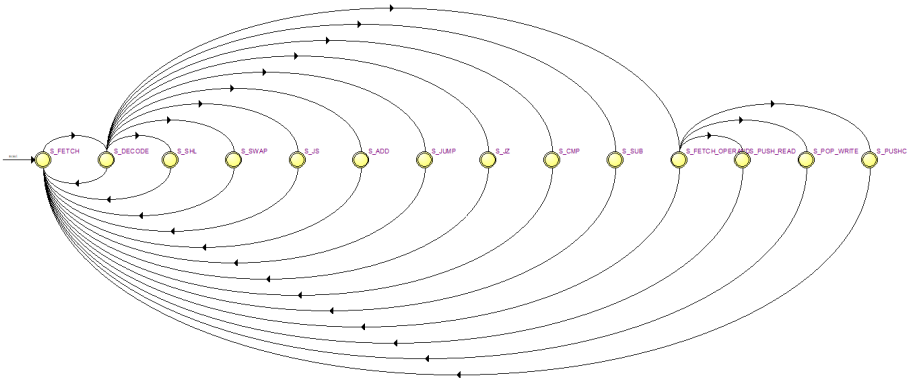


شکل ۱۶: خلاصه‌ی منابع مصرفی سامانه

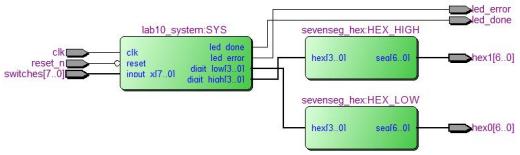
این طرح پرمصرف‌ترین طرح کلّ این مجموعه آزمایش‌هاست و دلیل آن در یک عدد نهفته است: مقدار بیت‌های حافظهٔ مصرف‌شده صفر گزارش شده، در حالی که طرح یک حافظهٔ ۲۵۶ خانه‌ای هشت‌بیتی دارد. یعنی سنتزکننده این حافظه را به‌جای یک بلوک حافظهٔ آماده، با $256 \times 8 = 2048$ فلیپ‌فلاپ ساخته است. همین یک آرایه، بیش از نود درصد کلّ 2,162 ثبات طرح را در بر می‌گیرد و بقیهٔ ثبات‌ها به پشته، شمارندهٔ برنامه، ثبات‌های دستور و عملوند، اشاره‌گر پشته، پرچم‌ها و درگاه‌های خروجی تعلّق دارند.

دلیل این انتخاب سنتزکننده آن است که در این توصیف، حافظه در یک بلوک مقداردهی اوّلیّه با برنامه پر می‌شود و هم‌زمان از دو مسیر ترکیبی خوانده می‌شود؛ چنین الگویی با ساختار بلوک حافظهٔ آمادهٔ قطعه نمی‌خواند. اگر هدف، کاهش مساحت بود، می‌شد حافظه را به‌گونه‌ای بازنویسی کرد که به بلوک حافظهٔ اختصاصی نگاشت شود.

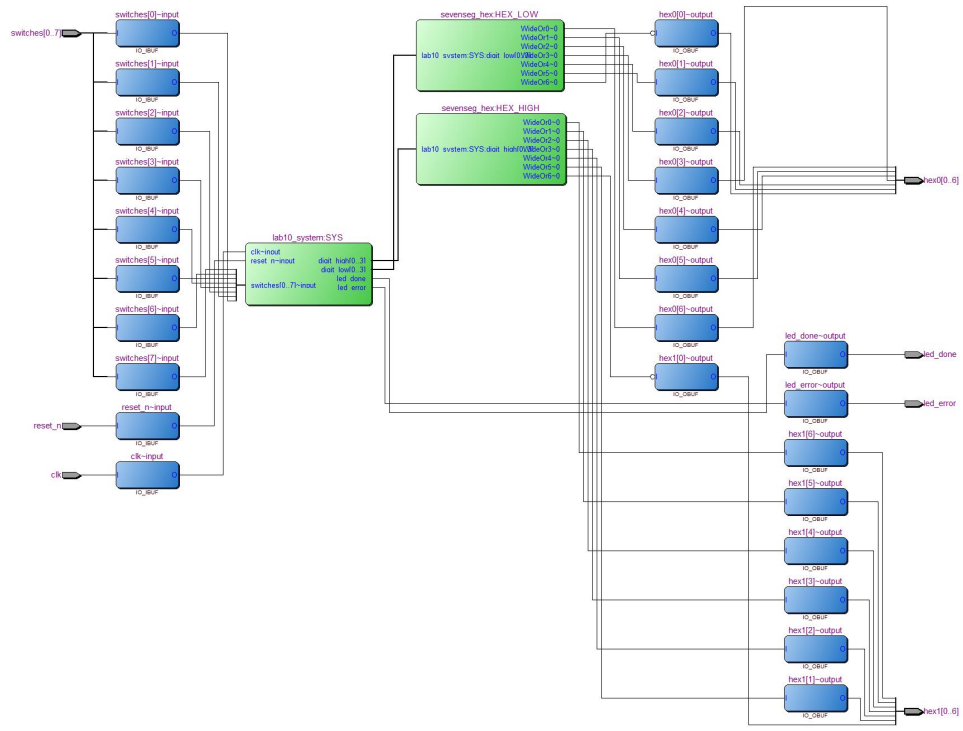
ب) نماهای ساختاری



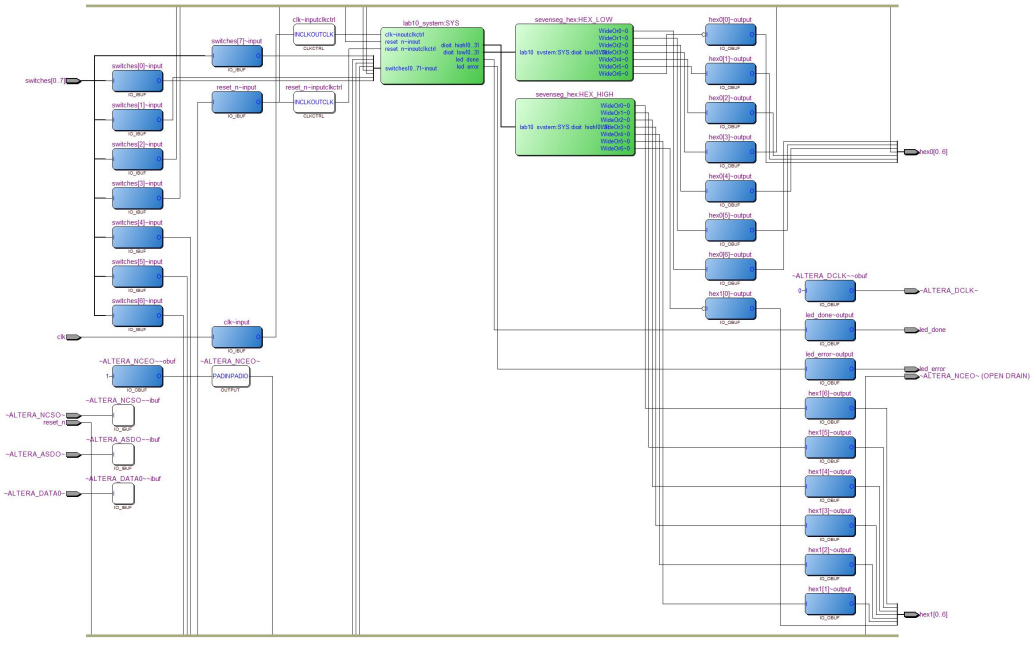
شکل ۱۷: نمای سطح انتقال ثبات‌های سامانه



شکل ۱۸: نمای سطح بالای طرح و اتّصال پردازنده به حافظه و درگاه‌ها



شکل ۱۹: نمای Technology Map Viewer (Post-Mapping)



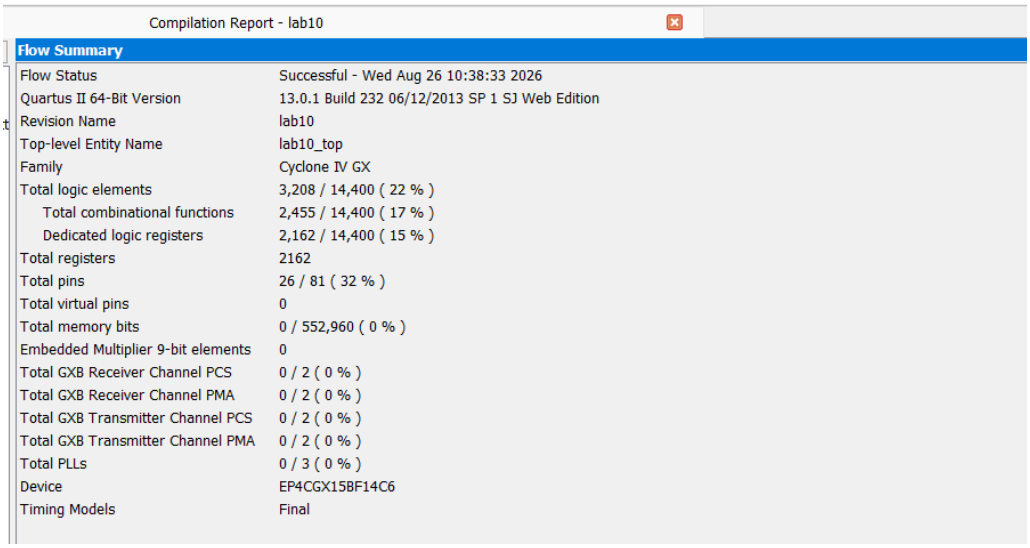
شکل ۲۰: نمای Technology Map Viewer (Post-Fitting)

ج) تحلیل زمانی

برای طرح یک پروندهٔ قیود طراحی[۱۴] نوشته شد که ساعتی با دورهٔ ۲۰ نانوثانیه، معادل ۵۰ مگاهرتز، تعریف می‌کند.

```
# 50 MHz
create_clock -name clk -period 20.000 [get_ports {clk}]
derive_clock_uncertainty
```

کد ۵: پرونده‌ی قیود طراحی



Compilation Report - lab10

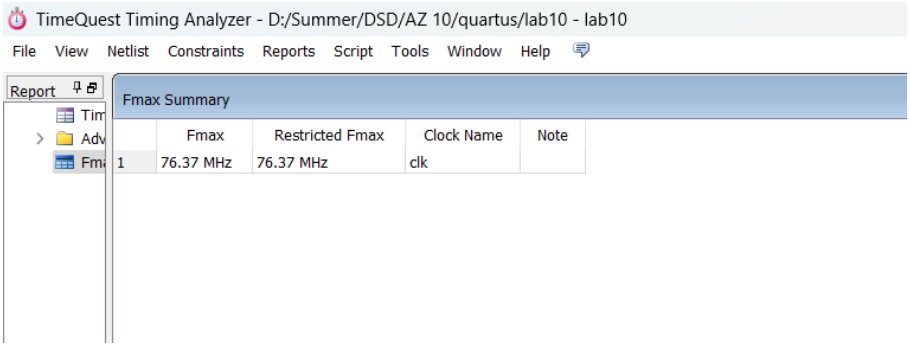
Flow Summary

Flow Status	Successful - Wed Aug 26 10:38:33 2026
Quartus II 64-Bit Version	13.0.1 Build 232 06/12/2013 SP 1 SJ Web Edition
Revision Name	lab10
Top-level Entity Name	lab10_top
Family	Cyclone IV GX
Total logic elements	3,208 / 14,400 (22 %)
Total combinational functions	2,455 / 14,400 (17 %)
Dedicated logic registers	2,162 / 14,400 (15 %)
Total registers	2162
Total pins	26 / 81 (32 %)
Total virtual pins	0
Total memory bits	0 / 552,960 (0 %)
Embedded Multiplier 9-bit elements	0
Total GXB Receiver Channel PCS	0 / 2 (0 %)
Total GXB Receiver Channel PMA	0 / 2 (0 %)
Total GXB Transmitter Channel PCS	0 / 2 (0 %)
Total GXB Transmitter Channel PMA	0 / 2 (0 %)
Total PLLs	0 / 3 (0 %)
Device	EP4CGX15BF14C6
Timing Models	Final

شکل ۲۱: خلاصه‌ی تحلیل‌گر زمانی

تحلیل‌گر زمانی[۱۵] در بدترین شرایط کاری حاشیهٔ برپایی[۱۶] 6.905 نانوثانیه گزارش می‌کند و مجموع حاشیه‌های منفی صفر است؛ یعنی قید ۵۰ مگاهرتزی با اطمینان برآورده می‌شود. بیشینه‌بسامد[۱۷] برابر 76.37 MHz است و بیشینه‌بسامد مقیّد نیز همین مقدار است.

[14] Synopsys Design Constraints
[15] TimeQuest Timing Analyzer
[16] Slack
[17] Maximum Frequency (Fmax)



شکل ۲۲: گزارش بیشینه‌بسامد سامانه

این بیشینه‌بسامد از بیشتر آزمایش‌های پیشین کمتر است و علّت آن همان حافظهٔ ساخته‌شده با ثبات است: خواندن از حافظه یک تسهیم‌کنندهٔ ۲۵۶ به یک می‌سازد که مسیر ترکیبی بلندی دارد و در مسیر بحرانی طرح قرار می‌گیرد.

با توجّه به برگزاری مجازی آزمایشگاه، ارزیابی این آزمایش بر پایهٔ شبیه‌سازی، سنتز و تحلیل زمانی نرم‌افزاری انجام شده و در پرونده‌های پیکربندی پروژه تخصیص پایه‌ای برای برد ثبت نشده است؛ ازاین‌رو ادّعایی دربارهٔ آزمون فیزیکی روی برد مطرح نمی‌شود.

۱۱.۱۰ نتایج

- پردازندهٔ پشته‌ای با هشت ثبات هشت‌بیتی، هشت دستور پایه و حافظهٔ ۲۵۶ خانه‌ای، مطابق دستورکار پیاده‌سازی شد.
- سه دستور SWAP، SHL و CMP افزوده شدند و هر سه با یک میز آزمون جداگانه آزموده گردیدند.
- ارتباط با ورودی و خروجی تنها از راه نگاشت در حافظه و با همان دستورهای PUSH و POP انجام شد.
- برنامهٔ زبان ماشین تابع چندبخشی خواسته‌شده را در سه شاخه محاسبه می‌کند و هر دو وضعیّت خطا را تشخیص می‌دهد.
- ضرب در دو با SHL و ضرب در سه با ترکیب SHL و ADD ساخته شد، بدون هیچ بلوک ضرب‌کننده‌ای.
- رقم پرارزش نمایشگر، در نبود دستورهای منطقی و شیفت راست، با تفریق پیاپی ساخته شد.
- بیشینهٔ عمق پشته برای هر ۲۵۶ ورودی ممکن برابر دو اندازه‌گیری شد؛ یعنی یک‌چهارم ظرفیّت پشتهٔ هشت‌تایی.

- درستی برنامه پیش از شبیه‌سازی سخت‌افزار، با یک شبیه‌ساز مستقل و برای هر ۲۵۶ ورودی وارسی شد.
- سنتز با 3,208 عنصر منطقی و 2,162 ثبات انجام شد و تحلیل زمانی حاشیهٔ مثبت 6.905 نانوثانیه و بیشینه‌بسامد 76.37 MHz داد.

نتیجه‌گیری

در این آزمایش یک پردازندهٔ کامل، از مجموعه دستور تا برنامهٔ زبان ماشین، ساخته و ارزیابی شد. آنچه این آزمایش را از آزمایش‌های پیشین متمایز می‌کند، دوسویه بودن طراحی است: مجموعه دستور تعیین می‌کند چه برنامه‌هایی نوشتنی‌اند، و نیازهای برنامه تعیین می‌کند چه دستورهایی باید افزوده شوند. هر سه دستور تازه از همین جهت زاده شدند؛ SHL چون ضرب لازم بود، و CMP چون مقایسه بدون از دست دادن عملوند لازم بود. محدودیّت‌های مجموعه دستور نیز خود را در جاهای دیگری نشان دادند: نبود دستور شیفت راست، ساختن رقم پرارزش نمایشگر را به یک حلقهٔ تفریق پیاپی تبدیل کرد. سرانجام، گزارش سنتز یادآور شد که سادگی یک معماری به معنای ارزان بودن پیاده‌سازی آن نیست؛ حافظهٔ ۲۵۶ خانه‌ای که در سطح توصیف تنها یک آرایه است، در سخت‌افزار بیش از دو هزار فلیپ‌فلاپ گرفت و مسیر بحرانی طرح را نیز همان ساخت.